

E9: 309 ADL 15-1-2021



Housekeeping

✳️ Midterm project III

✓ Evaluation after final exam (1st week of Feb) ✓

✳️ Final Exam (as per IISc schedule)

✓ Jan 23rd afternoon! ✓

✳️ Extra class (Friday 15, nov, 430pm)



Bayesian Deep Learning

✦ Goal -

- ➔ Show that the use of dropout (and its variants) in NNs can be interpreted as a Bayesian approximation of a well known probabilistic model.

✦ Goal -

- ➔ Develop tools for representing model uncertainty of existing dropout NNs – extracting information that has been thrown away so far. This mitigates the problem of representing model uncertainty in deep learning without sacrificing either computational complexity or test accuracy.



Definition of Gaussian process ^{$x(t)$} (Random process)

Discrete & Stationary

- * A Gaussian Process is a collection of random variables, any finite number of which have (consistent) joint Gaussian distributions.
- * A Gaussian process is fully specified by its mean function $m(x)$ and covariance function $k(x, x)$.

$$\underline{f \sim \mathcal{N}(m, k)}$$

- * This is a natural generalization of the Gaussian distribution whose mean and covariance is a vector and matrix, respectively. The Gaussian distribution is over vectors, whereas the Gaussian process is over functions.



Introduction to Gaussian processes

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}$$

$$\mathbf{f} = \begin{bmatrix} f(\mathbf{x}_1) \\ f(\mathbf{x}_2) \\ \vdots \\ f(\mathbf{x}_N) \end{bmatrix}$$

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

$$\mathbf{f} \sim \mathcal{N}(\boldsymbol{m}(\mathbf{x}), \boldsymbol{k}(\mathbf{x}, \mathbf{x}'))$$

✳ Mean will be function of $\mathbf{x}$ and variance will also be functions of two data points.



Gaussian process - Example

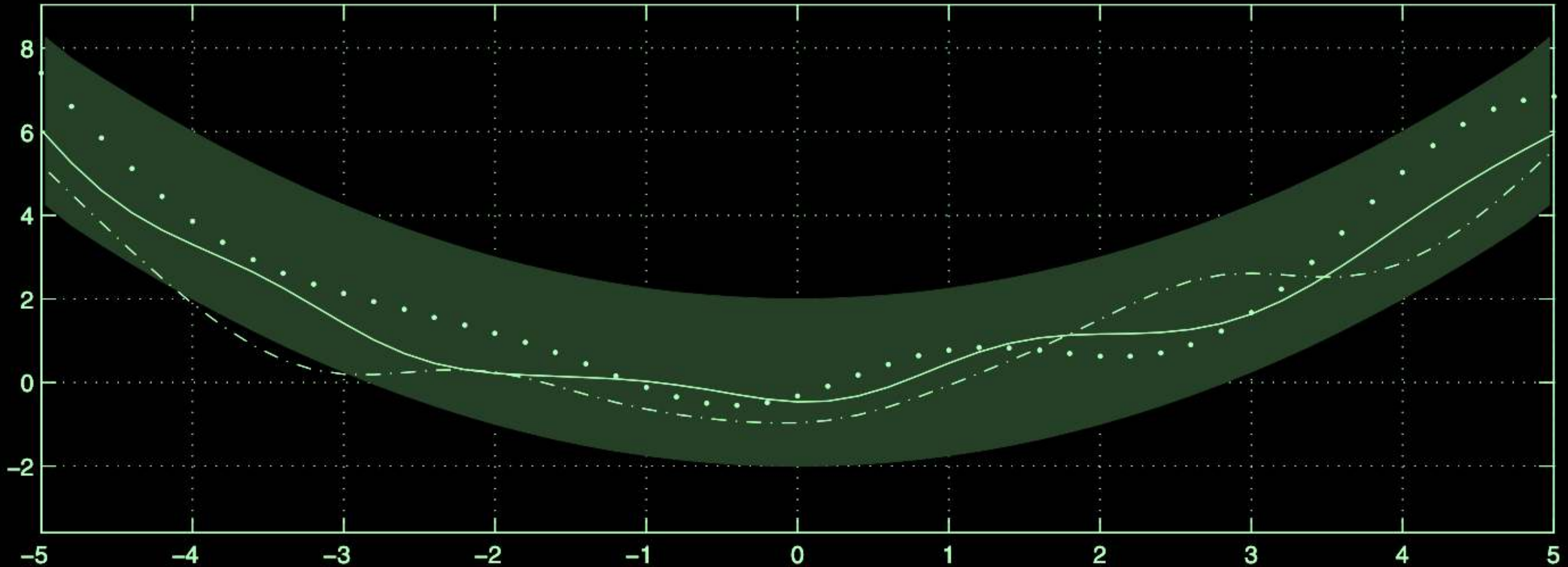


Fig. 1. Function values from three functions drawn at random from a GP as specified in Eq. (2). The dots are the values generated from Eq. (4), the two other curves have (less correctly) been drawn by connecting sampled points. The function values suggest a smooth underlying function; this is in fact a property of GPs with the squared exponential covariance function. The shaded grey area represent the 95% confidence intervals

Gaussian processes for Bayesian inference

- ✳ GP will be used as a prior for Bayesian inference.
- ✳ The prior does not depend on the training data, but specifies some properties of the functions.
- ✳ One of the primary goals computing the posterior is that it can be used to make predictions for unseen test cases.
- ✳ Let $\mathbf{f}$ be the known function values of the training cases, and let $\mathbf{f}_*$ be a set of function values corresponding to the test set inputs, $\mathbf{X}_*$.

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu \\ \mu_* \end{bmatrix}, \begin{bmatrix} \Sigma & \Sigma^* \\ \Sigma_*^T & \Sigma_{**} \end{bmatrix} \right)$$



Gaussian processes for Bayesian inference

✳ Now the quantity of interest is the posterior distribution (for function values)

$$\mathbf{f}_* | \mathbf{f} \sim \mathcal{N} \left(\mu_* + \Sigma_*^T \Sigma^{-1} (\mathbf{f} - \mu), \underbrace{\Sigma_{**}}_{\text{prior}} - \underbrace{\Sigma_*^T \Sigma^{-1} \Sigma_*} \right)$$

✳ Thus,

$$f | \mathcal{D} \sim \mathcal{GP}(m_D, k_D)$$

$$m_D(x) = m(x) + \Sigma(\mathbf{X}, x)^T \Sigma^{-1} (\mathbf{f} - \mathbf{m})$$

$$k_D(x, x') = \underbrace{k(x, x')} - \underbrace{\Sigma(\mathbf{X}, x)^T \Sigma^{-1} \Sigma(\mathbf{X}, x')}$$



Gaussian Processes

- ✳ where $\Sigma(X, x)$ is a vector of covariances between every training case and x . These are the central equations for Gaussian process predictions.
- ✳ Let's examine these equations for the posterior mean and covariance. Notice that the posterior variance $k_D(x, x)$ is equal to the prior variance $k(x, x)$ minus a positive term, which depends on the training inputs;
- ✳ thus the posterior variance is always smaller than the prior variance, since the data has given us some additional information



Allowing for noise in the model

- * Need to address one final issue: noise in the training outputs.
- * It is common to many applications of regression that there is noise in the observations⁶.
- * The most common assumption is that of additive i.i.d. Gaussian noise in the outputs.
- * In Gaussian process, the effect is that every $f(x)$ has a extra covariance with itself only (since the noise is assumed independent), with a magnitude equal to the noise variance:



Allowing for noise in the model

output of DNN

$$y(x) = \underbrace{f(x)}_{\text{output of DNN}} + \underbrace{\epsilon}_{\text{noise}}, \quad \epsilon \sim \mathcal{N}(0, \sigma_n^2)$$
$$\underbrace{f \sim \mathcal{GP}(m, k)}_{\text{prior}}, \quad \underbrace{y \sim \mathcal{GP}(m, k + \sigma_n^2 \delta_{i,i'})}_{\Sigma = k + \sigma_n^2 \mathbf{I}}$$

✳ Notice, that the indexes to the Kronecker's delta is the identify of the cases, i , and not the inputs $\mathbf{x}_i$; you may have several cases with identical inputs, but the noise on these cases is assumed to be independent.



Allowing for noise in the model



Fig. 2. Three functions drawn at random from the posterior, given 20 training data points, the $\mathcal{GP}$ as specified in Eq. (3) and a noise level of $\sigma_n = 0.7$. The shaded area gives the 95% confidence region. Compare with Figure 1 and note that the uncertainty goes down close to the observations

Dropout and its Bayesian Interpretation



Broad goal

* Interpretation of dropout as a Bayesian model

- ✓ offers an explanation to some of its properties, such as its ability to avoid over-fitting
- ✓ our insights allow us to treat NNs with dropout as fully Bayesian models, and obtain uncertainty estimates over their features.

* Mathematically,

- we will show that a deep neural network (NN) with arbitrary depth and non-linearities, with dropout applied before every weight layer, is mathematically equivalent to an approximation to the probabilistic deep Gaussian process model



Dropouts

Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning

Yarin Gal
Zoubin Ghahramani
University of Cambridge

YG279@CAM.AC.UK
ZG201@CAM.AC.UK



Dropout in NN

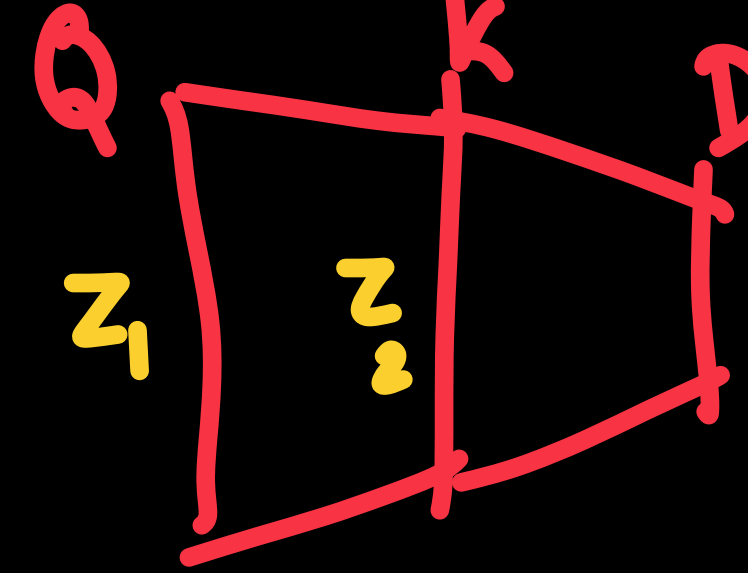
✳ Reviewing the dropout NN model quickly for the case of a single hidden layer NN. This is done for ease of notation, and the generalisation to multiple layers is straightforward.

✳ Denote by W_1, W_2 the weight matrices connecting the first layer to the hidden layer and connecting the hidden layer to the output layer respectively. These linearly transform the layers' inputs before applying some element-wise non-linearity $\sigma(\cdot)$. Denote by b the biases by which we shift the input of the non-linearity. We assume the model to output D dimensional vectors while its input is Q dimensional vectors, with K hidden units. Thus W_1 is a $Q \times K$ matrix, W_2 is a $K \times D$ matrix, and b is a K dimensional vector. A standard NN model would

$$\hat{y} = \sigma(\underbrace{xW_1 + b}_{\text{hidden layer input}})W_2$$



Dropout



- * Dropout is applied by sampling two binary vectors z_1, z_2 of dimensions Q and K respectively. The elements of the vectors are distributed according to a Bernoulli distribution with some parameter

$$\underline{p_i} \in \{0, 1\} \quad i = 1, 2$$

$$z_{1q} \sim \text{Bernoulli}(p_1)$$

$$z_{2k} \sim \text{Bernoulli}(p_2)$$

- * Given an input x , $(1 - p_1)$ proportion of the elements of the input are set to zero. (Drop-rate)

- * The output with dropout can be expressed as

$$\hat{y} = \sigma(x(\mathbf{Z}_1 \mathbf{W}_1) + \mathbf{b})(\mathbf{Z}_2 \mathbf{W}_2)$$

$$(\mathbf{Z}_1) = \text{diag}(\mathbf{z}_1) \quad (\mathbf{Z}_2) = \text{diag}(\mathbf{z}_2)$$



Loss function

* Loss in regression networks (MSE)

$$E = \frac{1}{2N} \sum_n ||\mathbf{y}_n - \hat{\mathbf{y}}_n||^2 \checkmark$$

* Loss in classification networks

$$\hat{p}_{nd} = \frac{\exp(\hat{y}_{nd})}{\sum_{d'} \exp(\hat{y}_{nd'})} \leftarrow \text{softmax}$$

$$E = -\frac{1}{N} \sum_n \log \hat{p}_{nc_n}$$

$$c_n \in [1 \dots D]$$

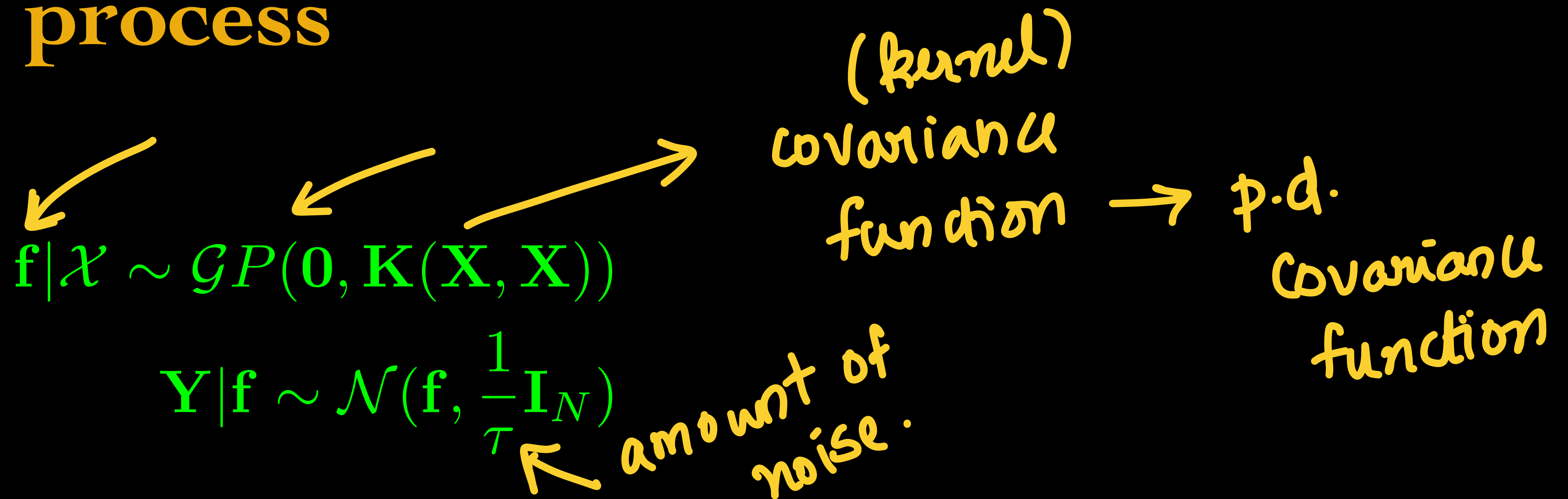
training data n has class c_n

* With L2 regularization, the total loss is

$$\mathcal{E}_{\text{dropout}} = \underbrace{E}_{\text{w}} + \underbrace{\lambda_1 ||\mathbf{W}_1||^2 + \lambda_2 ||\mathbf{W}_2||^2 + \lambda_3 ||\mathbf{b}||^2}_{\text{L2 regularization}}$$



Gaussian process

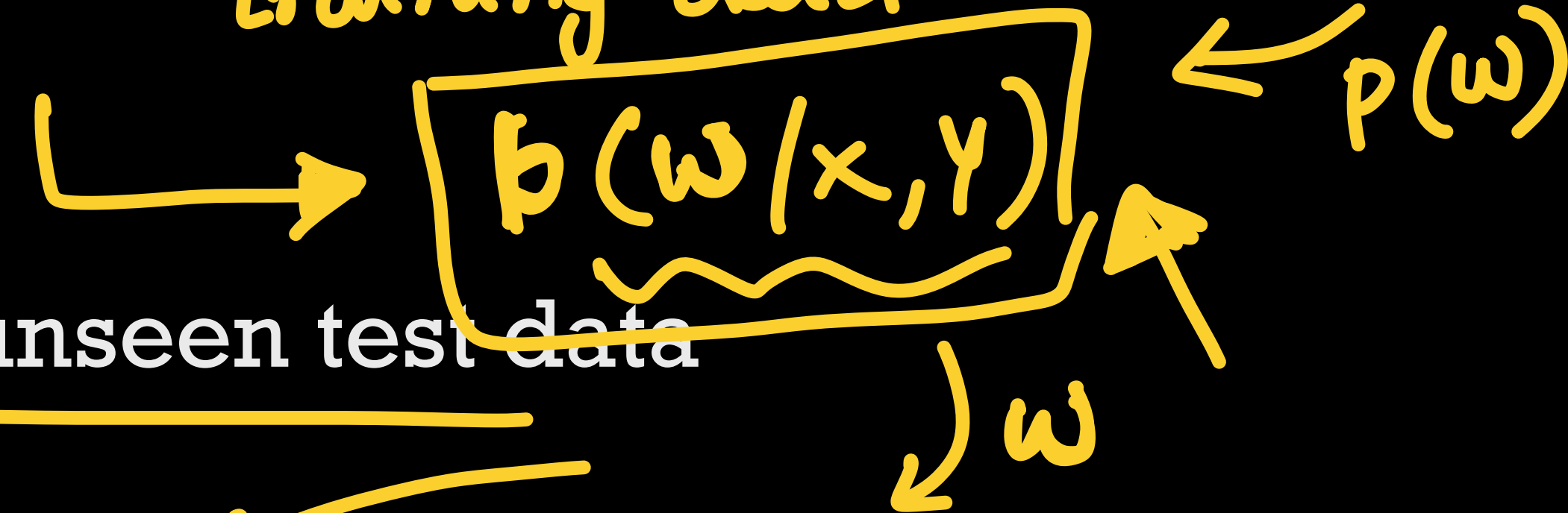


✳ To model the data we have to choose a covariance function $\mathbf{K}(\mathbf{X}_1, \mathbf{X}_2)$ for the Gaussian distribution. This function defines the (scalar) similarity between every pair of input points $\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j)$.

✳ Given a finite dataset of size N this function induces an $N \times N$ covariance matrix which we will denote $\mathbf{K} := \mathbf{K}(\mathbf{X}, \mathbf{X})$.

Variational Inference

x, y - training data



- * The output probability distribution on some unseen test data

$$p(y^* | x^*, X, Y) = \int p(y^* | x^*, \omega) p(\omega | X, Y) d\omega$$

The equation is annotated with yellow arrows: one points from x, y to x^* ; another points from x, y to X, Y ; a third points from ω in the integral to ω in the distribution $p(\omega | X, Y)$; and a fourth points from the boxed $p(\omega | x, y)$ in the diagram above to $p(\omega | X, Y)$.

- * condition the model on a finite set of random variables ω

$x, y \rightarrow \omega$
set of weights

✓ like the weights of the model.

- * The distribution $p(\omega | X, Y)$ cannot usually be evaluated analytically. Instead we define an approximating variational distribution $q(\omega)$

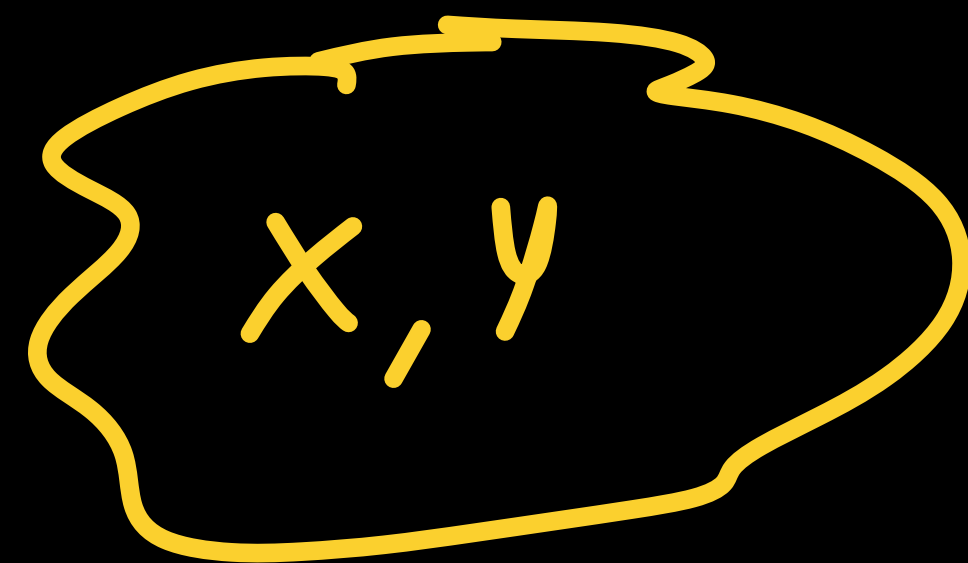


Two ingredients

(I)



prior on
model
weights



Training
data

$$\{ p(w/x, y) \}$$

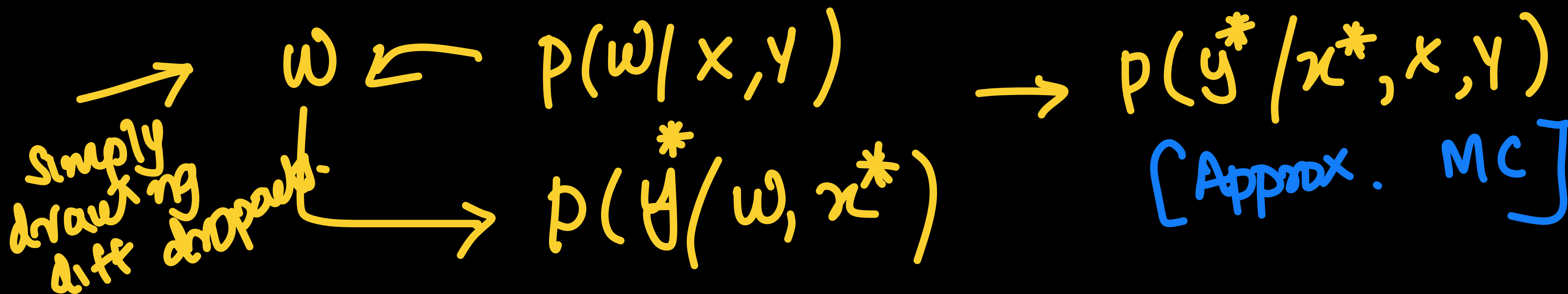
posterior
of model
weight

Approx (Variational learning)

(II)

Posterior on test data

[Uncertainty of the test data]



Forming a suitable approximation for the weight matrices

$$\mathbf{W}_2 = \mathbf{M}_1 \text{diag}(\mathbf{z}_1)$$

$$\mathbf{W}_2 = \mathbf{M}_2 \text{diag}(\mathbf{z}_2)$$

$q(w) \leftarrow$ variational learning

N - number of training pts

* $\mathbf{M}_1$ and $\mathbf{M}_2$ are full matrices and $\mathbf{z}_1$ and $\mathbf{z}_2$ are binary vectors with Bernoulli distribution parameterized using $\mathbf{p}_1$ and $\mathbf{p}_2$.

* In this scenario, maximizing the evidence lower bound gives

$$\underline{L_{GP}} \approx - \sum_{n=1}^N ||\mathbf{y}_n - \hat{\mathbf{y}}_n||^2 - \frac{p_1}{2} ||\mathbf{M}_1||^2 - \frac{p_2}{2} ||\mathbf{M}_2||^2$$

→ Very similar to the error function optimized in DNN training

$$\mathcal{E}_{dropout} = E + \lambda_1 ||\mathbf{W}_1||^2 + \lambda_2 ||\mathbf{W}_2||^2 + \lambda_3 ||\mathbf{b}||^2$$



Summary of previous slide

Bayesian learning

w — maximize the
posterior $p(w/x, Y)$
on training
data

→ Approx
VLB
(ELBO)



error function that is optimized
in a DNN with d2
weight reg. and dropout

Obtaining the model uncertainty

✳ Train the model using dropout and L2 regularization

✳ Under the assumed q distribution (z_1, z_2)

✓ Estimate the first order and second statistics of the output given the input MC

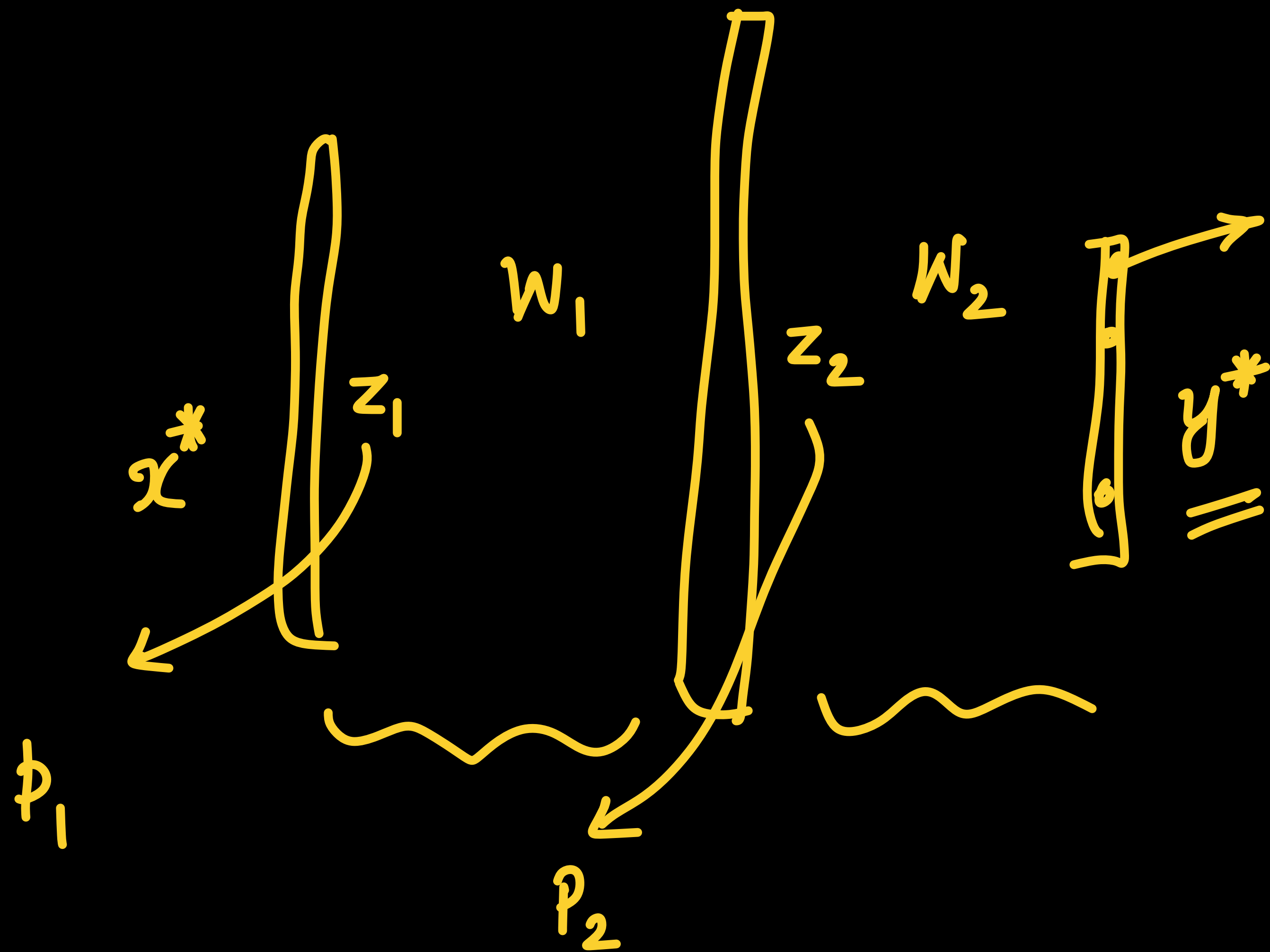
✓ Approximately equal to

○ First order and second statistic of the output with different dropouts for the given the input.

○ The first order moment is $\mathbb{E}_{q(y|x)} \approx \frac{1}{Q} \sum_{q=1}^Q \hat{y}(x, W_1^q, W_2^q)$ → Q random draws of model

$$p(y^* | x^*, x, y)$$





Application to uncertainty modeling in MNIST

✱ Train the MNIST model

➔ With dropout and regularization

✱ Obtain the output on a new test sample

$Q = 20$

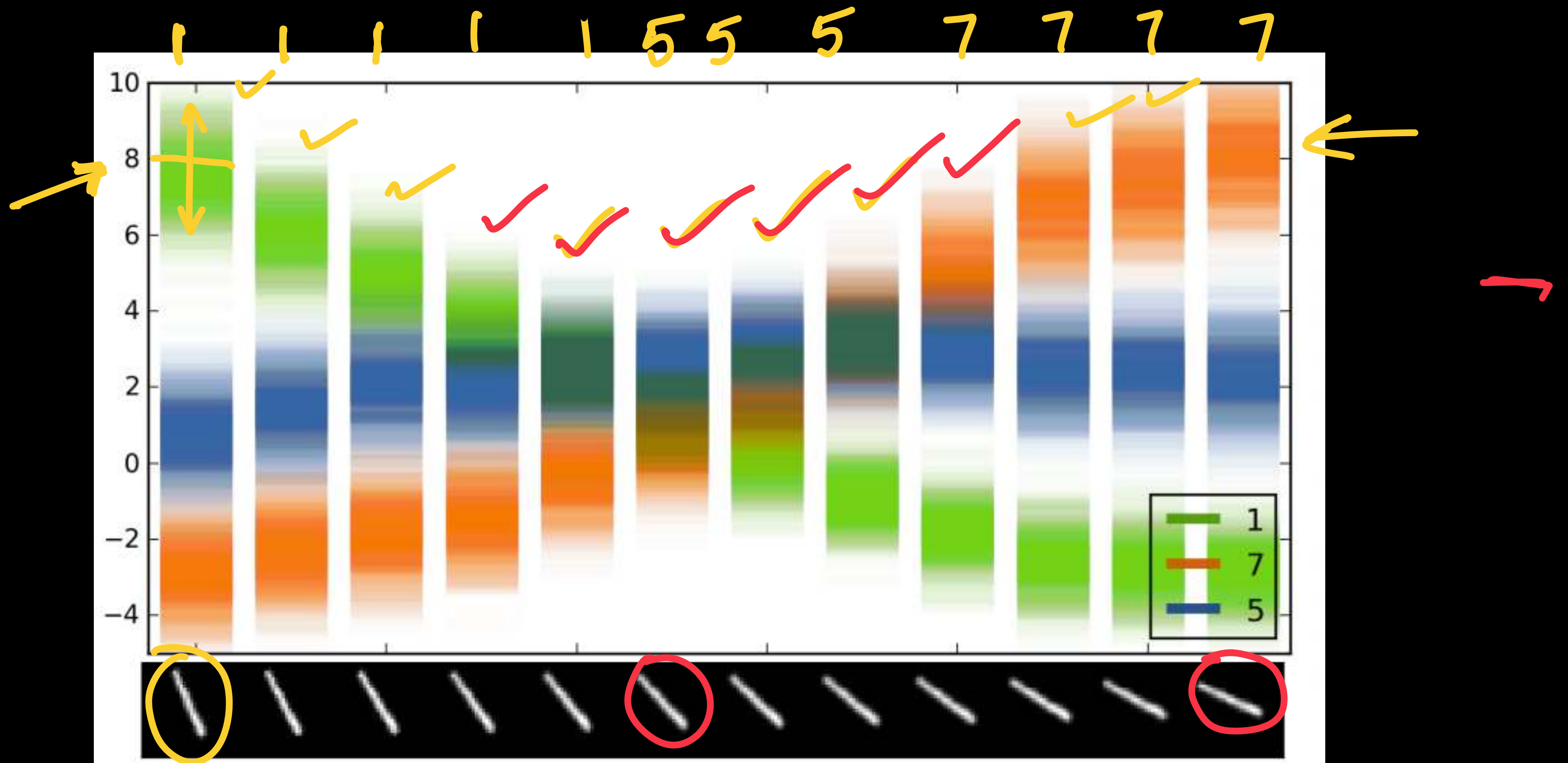
➔ Using different realizations of dropout on the test data

➔ Find the first and second moment of the output for each class

✓ Denoted as uncertainty in the model.

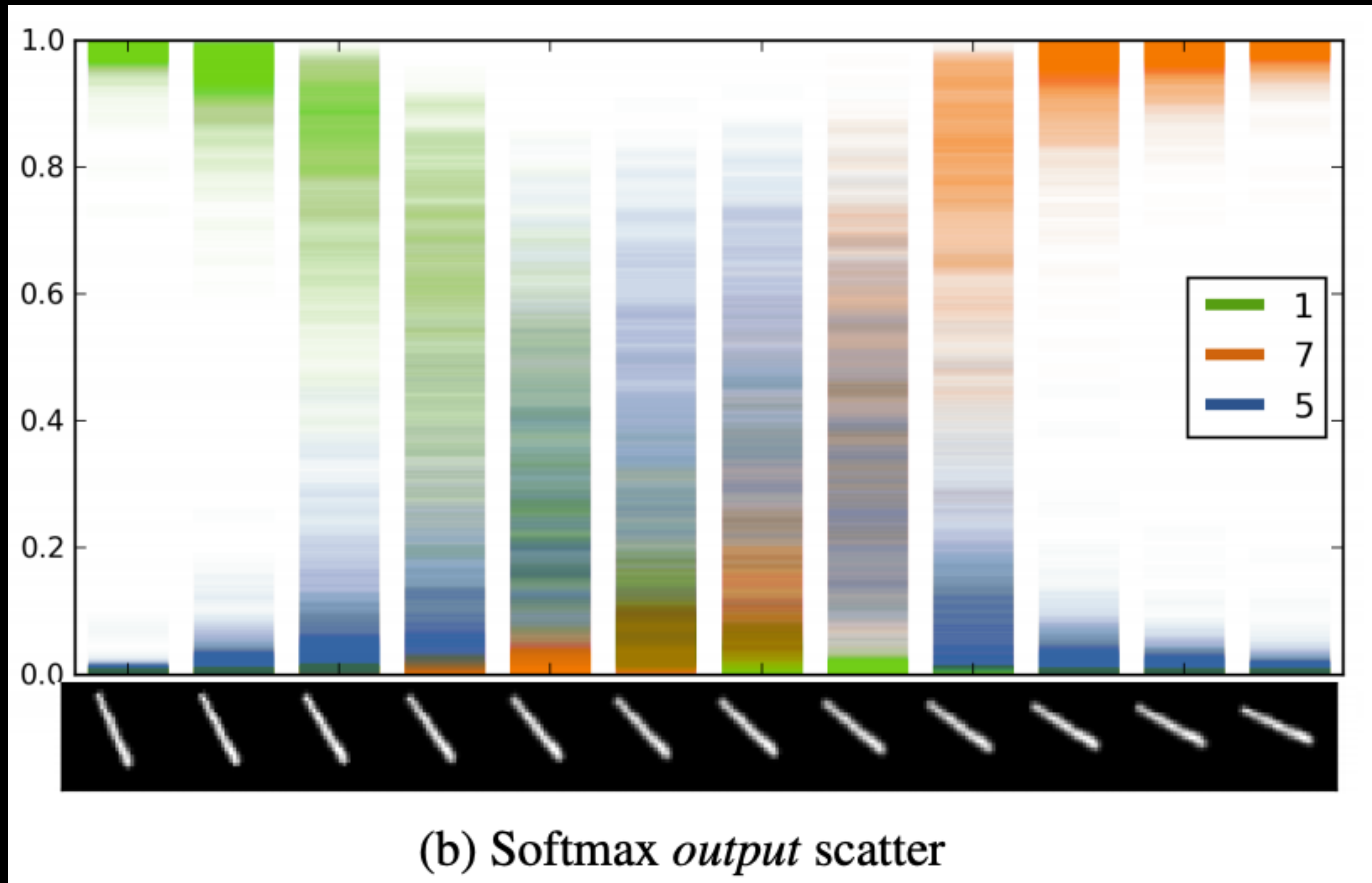


Uncertainty in MNIST



(a) Softmax input scatter

Uncertainty in the model output



Summary of ADL course ...

- ✳ **Visual and Time Series Modeling:** Semantic Models, Recurrent neural models and LSTM models, Encoder-decoder models, Attention models.
- ✳ **Unsupervised Learning:** Restricted Boltzmann Machines, Variational Autoencoders, Generative Adversarial Networks.
- ✳ **Representation Learning, Causality And Explainability:** t-SNE visualization, Hierarchical Representation, semantic embeddings, gradient and perturbation analysis, Topics in Explainable learning, Structural causal models. **Uncertainty modeling in deep learning.**
- ✳ **New Architectures:** Capsule networks, End-to-end models, Transformer Networks.
Graph networks
- ✳ **Applications:** Applications in in NLP, Speech, Image/Video domains in all modules.



Thanks

